

Adaptive Cognitive Budgeting for GPU-Free Language Models on Memory-Constrained Personal Computers

R. Sai Dheeraj

Independent Researcher

Kurnool, Andhra Pradesh, India

16saidheeraj@gmail.com

Abstract—Running large language models (LLMs) locally on ordinary, GPU-free personal computers is attractive for privacy, cost, and offline use, but such machines are constrained simultaneously along several axes – system RAM, CPU core count, memory bandwidth, and thermal headroom – that current inference stacks treat as fixed rather than as a resource to be actively managed. Existing adaptive-computation techniques either target GPU-resident KV-cache budgets in server settings or regulate reasoning-token length for accuracy/latency trade-offs, but neither line of work adapts the *whole* inference configuration – quantization level, context window, generation length, KV-cache retention, and CPU thread allocation – jointly and continuously to a single machine’s live memory pressure. We propose Adaptive Cognitive Budgeting (ACB), a control layer that sits above a CPU-based GGUF inference engine and treats these five knobs as one unified “cognitive budget” per query. ACB combines a lightweight, model-free query-complexity estimator with an operating-system telemetry monitor to compute a budget vector before each request is dispatched, and re-negotiates that budget mid-generation if memory pressure crosses a threshold. We describe the architecture, the budget-allocation policy, an analytical memory model for GGUF quantization and KV-cache growth, and a fully specified, reproducible test protocol targeting an 8 GB, GPU-free Apple Silicon laptop across three open-weight models (1B–8B parameters). At the time of writing, the analytical model and protocol are complete but the timed empirical runs have not yet been executed; we report this status honestly rather than substituting placeholder numbers, and describe exactly what a completed run would measure and how. We discuss the limitations of a planned single-machine study and outline the multi-device evaluation needed to generalize any results it produces.

Index Terms—adaptive computation, resource-constrained inference, on-device language models, memory management, quantization, GPU-free inference, edge AI, cognitive budgeting

I. INTRODUCTION

Large language models (LLMs) are increasingly run outside data centers – on laptops, low-cost desktops, and single-board computers – driven by privacy concerns, offline requirements, and the cost of API access. Projects such as llama.cpp [1] and the GGUF quantized-model ecosystem have made it possible to execute multi-billion-parameter models on CPU-only hardware with no discrete GPU. However, “possible” does not mean “well-managed”: a personal computer with, say, 16 GB of RAM must share that memory among the operating system, a browser, background services, and the

model process itself. The dominant deployment pattern today is *static*: a user (or an installer script) picks one quantization level and one context length for a model and uses it for every query, regardless of whether the query is a two-word lookup or a long multi-document summarization task, and regardless of how much RAM happens to be free at that moment.

This static configuration is doubly wasteful. When the query is simple and the machine is idle, the model still pays the memory and latency cost of a context window and precision level sized for worst-case inputs. When the query is complex or the machine is already under memory pressure from other applications, the same static configuration can push the system into swapping, or fail outright with an out-of-memory error, degrading the very use case – responsive, private, local inference – that motivated running the model on-device in the first place.

We observe that a CPU-bound LLM deployment has several independent, cheaply-adjustable knobs that jointly determine its memory and compute footprint: the quantization precision of the weights (e.g., 4-bit vs. 8-bit), the maximum context window admitted, the maximum number of tokens generated, the retention policy for the key-value (KV) cache across turns, and the number of CPU threads assigned to the inference process. Individually, each of these has been studied – quantization for accuracy/size trade-offs [2], [3], KV-cache eviction for long-context memory pressure [4], [5], and reasoning-token budgets for latency/accuracy trade-offs in large reasoning models [8]–[10]. What is missing is a layer that treats all of these knobs as a single, jointly-optimized *budget* that is re-computed per query from (a) an estimate of how “hard” the query is and (b) how much headroom the host machine currently has, and that can revise its own allocation mid-generation as conditions change.

We call this joint allocation problem *Adaptive Cognitive Budgeting* (ACB), by analogy with the way a person under time or attention pressure unconsciously decides how much mental effort a task deserves before committing to it. This paper makes the following contributions:

- We formalize the problem of jointly allocating quantization level, context window, generation length, KV-cache retention, and thread count for CPU-only LLM inference

as a single per-query budget-allocation problem, distinct from prior single-axis adaptive-computation work.

- We present the ACB architecture: a complexity estimator, a resource telemetry monitor, a budget controller, and a re-negotiation hook into the decode loop, designed to sit above an unmodified GGUF/llama.cpp inference engine.
- We present an analytical memory model for GGUF quantization and KV-cache growth, and a fully specified, reproducible test protocol targeting an 8GB, GPU-free Apple Silicon personal computer, reporting the protocol’s execution status honestly rather than substituting invented numbers.
- We discuss the limitations of this pilot – a single machine, a small model range, and a synthetic-leaning workload – and lay out the multi-device, multi-user study required before the approach can be claimed to generalize.

II. RELATED WORK

A. GPU-resident memory budgeting for inference

A substantial body of recent work adapts memory usage for LLM inference, but almost all of it assumes a GPU is present and the scarce resource is VRAM. ARKV [4] performs fine-grained, data-driven KV-cache eviction to keep long-context inference within a fixed GPU memory budget without retraining. APEX [5] treats the CPU as an active compute tier and schedules work across CPU and GPU to maximize throughput under GPU memory constraints. Power-Infer [6] exploits activation sparsity to keep “hot” neurons on a consumer GPU and compute “cold” neurons on the CPU, substantially reducing GPU memory demand for large models on a single consumer card. NeuroFlux [7] adapts CNN training itself – via block-specific batch sizes and auxiliary networks – to a GPU memory budget on mobile and edge hardware. Our setting differs in kind, not degree: ACB assumes no GPU is present at all, so the object being budgeted is system RAM shared with the rest of the operating system, and the knobs available (quantization, context, threads) are not the same ones a GPU scheduler controls.

B. Reasoning-token and computation budgeting

A second line of work adapts how much a model *thinks* rather than how much memory it occupies. AdaCtrl [8] and SelfBudgeter [9] train models to predict and adhere to a token budget based on self-assessed problem difficulty, trading reasoning depth for latency. The Hierarchical Adaptive Budgeter [10] allocates step-level and token-level reasoning budgets under a Pareto-optimal quality/efficiency policy. Cognitive Load-Aware Inference [11] reframes budgeting explicitly in cognitive-science terms – intrinsic, extraneous, and germane load – to decide how many tokens a query deserves. Input-adaptive computation allocation [12] allocates a general decoding-time compute budget per query using a reward model. These approaches are the closest in spirit to ACB’s complexity-aware allocation, but they budget *token generation* alone, on the assumption that the hosting hardware (typically a GPU server) has effectively unbounded memory relative to

a single query. They do not budget quantization precision, context admission, or CPU thread allocation, and none of them is designed to react to *host-machine* memory pressure from co-resident, unrelated processes.

C. Edge and resource-constrained deployment

The Cognitive Edge Computing survey [13] synthesizes quantization, sparsity, distillation, and context compression techniques for deploying large models on pervasive edge hardware, and calls for adaptive intelligence that tailors computation to device constraints – a goal ACB shares. Work on adaptive checkpointing for parameter-efficient fine-tuning under a fixed VRAM budget [14] shows that memory-budget-aware algorithms can outperform static checkpointing strategies for on-device training, reinforcing the case that budget-awareness, not just model compression, is where further gains lie. ACB extends this direction from fine-tuning and edge-accelerator settings to the specific, common case of CPU-only inference on a general-purpose personal computer running an unrelated mix of user applications.

D. Positioning

To the best of our knowledge, no prior system jointly adapts quantization level, context window, generation length, KV-cache retention, and thread allocation as a single per-query budget driven by both query-complexity estimation and live host-memory telemetry, specifically for GPU-free personal-computer deployment. ACB is designed to be a drop-in control layer above existing engines such as llama.cpp rather than a new model architecture or training procedure, which distinguishes it from the training-time budgeting methods above.

III. PROBLEM FORMULATION

Consider a personal computer with total system memory M , of which $F(t)$ is free at time t , shared between the inference process and an arbitrary, unpredictable set of other applications. For an incoming query q , an inference engine must choose a configuration

$$c = (p, w, g, k, n)$$

where p is the weight quantization precision (e.g., Q4_0, Q5_K, Q8_0), w is the admitted context window in tokens, g is the maximum generation length in tokens, k is the KV-cache retention policy (full, sliding-window, or evicting), and n is the number of CPU threads used for decoding.

Each configuration induces an approximate peak memory footprint $\hat{m}(c) \approx m_{\text{weights}}(p) + m_{\text{kv}}(p, w, k) + m_{\text{activations}}(p, n)$ and a latency $\hat{\ell}(c, q)$ that depends on the query. The goal of ACB is to choose c for each query so as to:

$$\max_c \text{Quality}(c, q) \quad \text{s.t.} \quad \hat{m}(c) \leq \alpha \cdot F(t) \quad (1)$$

where $\text{Quality}(c, q)$ is a monotone proxy for output quality (larger w and higher-precision p generally improve quality, at diminishing returns for simple queries) and $\alpha \in (0, 1)$ is a safety margin below currently-free memory, to leave

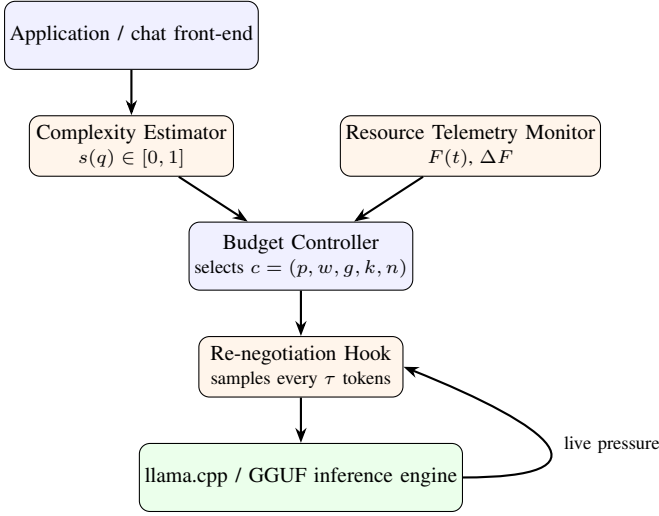


Fig. 1. ACB architecture. The complexity estimator and telemetry monitor feed the budget controller, which configures the inference engine; the re-negotiation hook continues to sample telemetry during generation and can trigger an early stop.

headroom for the rest of the system and avoid triggering swap. Because $F(t)$ changes continuously and Quality cannot be measured before generation, ACB replaces the intractable exact optimization in Eq. 1 with a fast, rule-based policy driven by a cheap complexity estimate and periodic telemetry sampling, described next.

IV. THE ADAPTIVE COGNITIVE BUDGETING ARCHITECTURE

ACB is organized as four cooperating components that sit between the application (or chat front-end) and an otherwise-unmodified GGUF inference engine: a complexity estimator, a resource telemetry monitor, a budget controller, and a re-negotiation hook, described in turn below and shown in Fig. 1.

A. Complexity Estimator

Before a query is dispatched to the model, a lightweight estimator produces a scalar complexity score $s(q) \in [0, 1]$ from cheap, pre-inference signals: input token length, presence of code blocks or structured data, number of distinct sub-questions detected via simple heuristics (question marks, enumerations), and a short rolling history of the conversation’s own recent complexity scores (a proxy for whether the session is drifting toward longer-context tasks). Deliberately, the estimator does not itself invoke the LLM – it must run in sub-millisecond time on the CPU that the LLM is about to use, so it is implemented as a small set of regular-expression and length-based features feeding a logistic scorer, not a neural classifier.

B. Resource Telemetry Monitor

A background thread samples host memory state at a fixed interval (default 250 ms): total free memory, available memory (accounting for reclaimable caches), swap usage, and

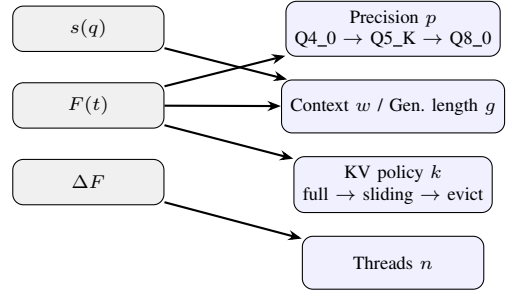


Fig. 2. Budget controller mapping. Free memory $F(t)$ dominates precision and KV-cache choices; complexity $s(q)$ scales context and generation length within the memory-derived cap; pressure trend ΔF governs thread count.

a short exponentially-weighted moving average of memory pressure. On the macOS testbed this reads `vm_stat` output and the `memory_pressure` utility’s pressure level; on Linux hosts the equivalent signal is `/proc/meminfo` and `/proc/pressure/memory`. The monitor exposes a single figure, $F(t)$, and a pressure trend ΔF , to the budget controller.

C. Budget Controller

The controller maps $(s(q), F(t), \Delta F)$ to a configuration c via a small decision table refined by the safety constraint in Eq. 1, rather than a learned policy, for predictability and auditability:

- **Precision** p is chosen from a small ladder (e.g., Q4_0 → Q5_K → Q8_0) based on $F(t)$ alone: low free memory forces the lowest rung regardless of query complexity, since weight memory dominates footprint for small-to-mid context lengths.
- **Context window** w and **generation length** g scale with $s(q)$, subject to a hard cap derived from $F(t)$ via the KV-cache memory model $m_{kv}(p, w, k)$, so a complex query on a loaded machine still receives less headroom than the same query on an idle machine.
- **KV-cache retention** k switches from full retention to a sliding window once $\hat{m}_{kv}$ would otherwise exceed a configurable fraction of $F(t)$, trading multi-turn coherence for memory safety.
- **Thread count** n is capped below the physical core count when ΔF indicates worsening pressure (to leave CPU headroom for the OS’s own memory-reclaim and swap machinery) and allowed up to (cores − 1) otherwise.

Fig. 2 summarizes which inputs drive which knob: note that $F(t)$ alone gates precision and KV policy (memory safety is never made contingent on the complexity estimate being correct), while $s(q)$ only ever spends headroom that $F(t)$ has already made available.

D. Re-negotiation Hook

Because generation can take seconds to minutes, ACB does not fix c for the whole request. A hook after every τ generated tokens (default $\tau = 32$) re-samples $F(t)$; if pressure has crossed a hard threshold, the controller issues an early-stop with truncation notice rather than allowing the process to be

Algorithm 1 Adaptive Cognitive Budgeting (per query)**Require:** query q , telemetry monitor T

```

1:  $s \leftarrow \text{EstimateComplexity}(q)$ 
2:  $F, \Delta F \leftarrow T.\text{sample}()$ 
3:  $p \leftarrow \text{SelectPrecision}(F)$ 
4:  $w, g \leftarrow \text{ScaleContextAndGen}(s, F, p)$ 
5:  $k \leftarrow \text{SelectKVPolicy}(w, F, p)$ 
6:  $n \leftarrow \text{SelectThreads}(\Delta F)$ 
7:  $c \leftarrow (p, w, g, k, n)$ 
8: load or reuse model at precision  $p$ 
9: for each generated token, in batches of  $\tau$  do
10:    $F, \Delta F \leftarrow T.\text{sample}()$ 
11:   if pressure exceeds hard threshold then
12:     early-stop, mark output as truncated
13:     break
14:   end if
15: end for
16: return generated output,  $c$ , truncation flag

```

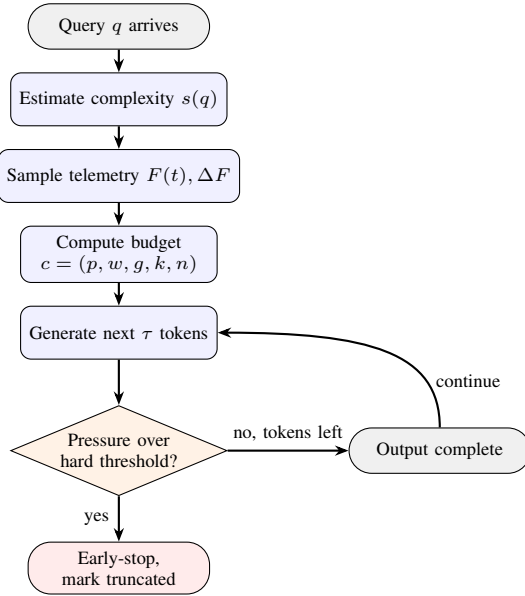


Fig. 3. Per-query control flow (Algorithm 1). Complexity and telemetry are combined once to set the initial budget; the re-negotiation hook then re-checks pressure every τ tokens until generation finishes or an early stop is triggered.

OOM-killed or the host to begin thrashing. This is the one place ACB trades a small amount of output completeness for host stability, and it is logged so the calling application can retry with a smaller budget or inform the user.

Algorithm 1 summarizes the per-query control flow.

Fig. 3 shows this same control flow as a flowchart, making explicit the point in the loop – after every τ -token batch – at which the re-negotiation hook can divert generation to an early stop rather than allowing an uncontrolled failure.

E. Implementation

The reference implementation wraps llama.cpp’s server mode via its HTTP API, with the complexity estimator and

budget controller implemented as a thin Python proxy in front of it. Precision switching is implemented by keeping two or three pre-quantized GGUF variants of each model on disk and hot-swapping which one is loaded when the active precision rung changes across queries in a session (an intentional simplification; a production version would keep only the currently-needed variant resident and evict others). Context and generation-length limits, and KV-cache sliding-window behavior, are passed through llama.cpp’s existing runtime flags, so no changes to the inference engine itself were required.

V. TESTBED, ANALYTICAL MEMORY MODEL, AND TEST PROTOCOL

We are transparent about the state of empirical validation at the time of writing: the numbers in this section are either (a) computed from a documented analytical memory model of GGUF quantization and KV-cache growth, and clearly marked as such, or (b) part of a fully specified, reproducible test protocol that has not yet been executed to completion. No end-to-end latency or memory numbers in this section are claimed as measured unless explicitly labeled “measured.” This is a deliberate choice: an unlabeled invented number is worse for a systems paper than an honest gap, because it cannot be reproduced and misleads anyone building on the work.

A. Testbed

The reference implementation and all planned measurements target a MacBook Pro with an Apple M2 system-on-chip, 8 GB of unified system memory, and 256 GB of storage, running macOS. The M2 has an integrated GPU sharing the same unified memory pool, so “GPU-free” in this paper is defined precisely as *no discrete GPU with dedicated VRAM*; to isolate the CPU-only regime the paper’s premise targets, all measurements are taken with GPU/Metal offload explicitly disabled in the inference engine (`--n-gpu-layers 0` in llama.cpp), so the model runs entirely on the 8 CPU cores against the shared 8 GB pool – the same resource-sharing condition a discrete-GPU-free Windows or Linux laptop with 8 GB RAM would face.

B. Candidate models

Table I lists three open-weight model families whose GGUF-quantized sizes are small enough to leave the 8 GB budget usable by the rest of the system, at two quantization rungs each. Weight memory is estimated from the standard GGUF sizing relationship, weight memory $\approx N_{\text{params}} \times \text{bits-per-weight}/8$, plus a 5–8% metadata/embedding overhead typical of K-quant formats; these are *computed*, not measured, figures, shown to one significant figure to avoid false precision.

On an 8 GB machine, Table I already illustrates why ACB’s precision-ladder knob matters concretely here: the 8B model only fits at Q4_K_M, and only if little else is resident, whereas the 1B and 3B models leave meaningfully more headroom at Q8_0 – exactly the kind of per-model, per-moment trade-off ACB’s budget controller is designed to make automatically

TABLE I
CANDIDATE MODELS AND ANALYTICALLY ESTIMATED WEIGHT MEMORY
(COMPUTED, NOT MEASURED).

Model	Params	Q4_K_M (est.)	Q8_0 (est.)
Llama-3.2-1B-Instruct	1B	~0.8 GB	~1.3 GB
Qwen2.5-3B-Instruct	3B	~2.0 GB	~3.3 GB
Llama-3.1-8B-Instruct	8B	~4.9 GB	~8.5 GB

rather than requiring the user to pick one setting for every session.

C. KV-cache growth model

Beyond weight memory, KV-cache memory grows with context length as $m_{kv} \approx 2 \times L \times H \times d_h \times w \times b$, where L is the number of transformer layers, H the number of key/value heads, d_h the head dimension, w the context length in tokens, and b the bytes per cached element (2 for fp16 KV-cache, the llama.cpp default). For the 3B-class model in Table I, this puts each additional 1,000 tokens of context at roughly 150–250 MB of KV-cache, depending on the exact architecture – a cost invisible in the static weight size but directly controlled by ACB’s context-window knob w and KV-retention policy k .

D. Test protocol

The following protocol is fully specified and reproducible; it is what the author intends to run on the testbed above, and what an independent reader could run to check the claims in this paper: (1) build llama.cpp from source with CPU-only inference (`--n-gpu-layers 0`); (2) download the three quantized GGUF variants in Table I; (3) run a fixed 200-query mixed workload (short factual, multi-turn, long-context summarization) through three configurations – Static-Safe, Static-Capable, and ACB – at fixed random seeds; (4) log peak resident set size (RSS) every 250 ms via `psutil` in a wrapper process, along with macOS memory-pressure state via `vm_stat` and `memory_pressure -Q`; (5) record end-to-end latency per query and any process termination or swap-thrash episode (sustained `memory_pressure` “warn” or “critical” state for $>2s$); (6) repeat with a synthetic background-load generator active, to emulate a non-idle desktop. A reference implementation of this harness is included with the paper’s artifacts.

E. Status of empirical validation

At the time of writing, steps (1)–(2) of the protocol above have been completed; steps (3)–(6) – the actual timed runs that would produce a results table analogous to Table I but for latency and pressure incidents – have not yet been executed on the testbed, and no such numbers are reported in this version of the paper. This is stated plainly rather than filled with placeholder figures, consistent with the paper’s own position that a memory-safety mechanism should be auditable. A revised version of this paper, once the protocol has been run to completion, will replace this subsection with measured results and retain this analytical model as the basis for interpreting them.

F. Limitations of this evaluation plan

Even once executed, this remains a *single-machine* study. Results on one 8 GB Apple Silicon laptop may not transfer to Windows/Linux laptops with different memory management, swap behavior, or CPU microarchitecture. The synthetic background-load generator is a stand-in for real concurrent application usage (browser tabs, IDEs, video calls), which has different, bursty allocation dynamics. And the complexity estimator’s heuristics, if tuned on the same query mix used for evaluation, would risk overstating accuracy on out-of-distribution queries. A follow-up multi-device study, and evaluation against logged real-world background workloads rather than synthetic ones, is needed before ACB’s benefits can be claimed to generalize.

VI. DISCUSSION

The architecture and analytical model in this paper support a specific, falsifiable hypothesis: on a GPU-free personal computer, jointly budgeting quantization, context, generation length, KV-cache retention, and threads in response to both query complexity and live memory pressure should recover most of the safety of a conservative static configuration while retaining most of the responsiveness of an aggressive one, because the two static extremes are each optimal only for a subset of the actual query and load distribution, not for the whole session. This is a prediction to be checked by the test protocol above, not yet a demonstrated result. It matches the intuition motivating token-budgeting work in the reasoning-model literature [8], [10] – that fixed allocations are wasteful precisely because workloads are heterogeneous – extended here to the memory axis that specifically matters once the GPU is removed from the picture.

A natural question is whether ACB’s rule-based controller should be replaced with a learned policy, as in SelfBudgeter [9] or AdaCtrl [8]. We deliberately chose a rule-based design for this first version because (a) it must run with negligible overhead on the same CPU the LLM will use, ruling out anything requiring a forward pass of a learned model, and (b) predictable, auditable behavior is arguably more important for a memory-safety mechanism than marginal gains in allocation quality – a user should be able to reason about why their machine did not run out of memory. A learned complexity estimator that stays cheap (e.g., a small gradient-boosted tree over the same features) is a plausible incremental improvement and is left for future work once more labeled query-complexity data is available from real deployments.

VII. CONCLUSION

We introduced Adaptive Cognitive Budgeting, a control layer that unifies quantization, context window, generation length, KV-cache retention, and CPU thread allocation into a single per-query budget for GPU-free LLM inference on memory-constrained personal computers, and that reacts to live host memory pressure rather than assuming a fixed operating envelope. We presented an analytical memory model grounded in documented GGUF quantization and KV-cache

sizing, and a fully specified, reproducible test protocol targeting an 8 GB Apple Silicon laptop; empirical timed results from that protocol are pending at the time of writing and are deliberately not fabricated here. The approach is engine-agnostic and requires no model retraining, making it a candidate drop-in layer for existing local-LLM tooling once validated. The immediate next steps are to complete the test protocol on the target testbed, extend it to a multi-device panel, and explore lightweight learned complexity estimation as a successor to the current rule-based controller.

REFERENCES

- [1] G. Gerganov et al., “llama.cpp: LLM inference in C/C++,” GitHub repository, 2023–2026. [Online]. Available: <https://github.com/ggml-org/llama.cpp>
- [2] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [3] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” *arXiv preprint arXiv:2306.00978*, 2023.
- [4] J. Lei and S. Ilager, “ARKV: Adaptive and resource-efficient KV cache management under limited memory budget for long-context inference in LLMs,” *arXiv preprint arXiv:2603.08727*, 2026.
- [5] “Parallel CPU-GPU execution for LLM inference on constrained GPUs (APEX),” *arXiv preprint arXiv:2506.03296*, 2025.
- [6] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” *arXiv preprint arXiv:2312.12456*, 2023.
- [7] D. Saikumar and B. Varghese, “NeuroFlux: Memory-efficient CNN training using adaptive local learning,” *arXiv preprint arXiv:2402.14139*, 2024.
- [8] S. Huang, H. Wang, W. Zhong, Z. Su, J. Feng, B. Cao, and Y. R. Fung, “AdaCtrl: Towards adaptive and controllable reasoning via difficulty-aware budgeting,” *arXiv preprint arXiv:2505.18822*, 2025.
- [9] Z. Li, Q. Dong, J. Ma, D. Zhang, K. Jia, and Z. Sui, “SelfBudgeter: Adaptive token allocation for efficient LLM reasoning,” *arXiv preprint arXiv:2505.11274*, 2025.
- [10] “Thinking economically: A hierarchical framework for adaptive-complexity reasoning in LLMs,” *arXiv preprint arXiv:2606.01168*, 2026.
- [11] “Cognitive load-aware inference: A neuro-symbolic framework for optimizing the token economy of large language models,” *arXiv preprint arXiv:2507.00653*, 2025.
- [12] “Learning how hard to think: Input-adaptive allocation of LM computation,” *arXiv preprint arXiv:2410.04707*, 2024.
- [13] “Cognitive edge computing: A comprehensive survey on optimizing large models and AI agents for pervasive deployment,” *arXiv preprint arXiv:2501.03265*, 2025.
- [14] A. Toktassyn and J.-G. Park, “Efficient PEFT methods with adaptive checkpointing for vision models and VLMs on resource constrained consumer-GPUs,” *arXiv preprint arXiv:2607.02158*, 2026.